Семинар 3. Системная архитектура распределенных систем: Docker, Linux и т.п.

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Первое домашнее задание – жесткий дедлайн в 23:59 19.03.2026
  - Всего 10 баллов: автотесты и корректность кода – допуск (ровно фиксированные 5 баллов), оцениваются ответы на вопросы от 0 до 5 баллов
  - Ответы на вопросы должны быть подробными, с примерами и объяснениями, а не просто «да» или «нет»
  - Учитывая невозможность проверить ответы на использование GPT, требования к качеству ответов высокие
  - Если есть замечания, можно будет исправить в течение недели после дедлайна без потери баллов
  - Для посылок после дедлайна – штраф 50%
  - (!) Не забывайте указать ФИО в своем Pull Request
- Всего будет ± 5 обязательных домашних заданий (± 10 баллов за каждое), которые будут выходить примерно раз в 2 недели
  - **A – «5»** – $\geq 90\%$ от общей суммы обязательных баллов
  - **B/C – «4»** – $\geq 75\%$ от общей суммы обязательных баллов
  - **D/E – «3»** – $\geq 60\%$ от общей суммы обязательных баллов
  - **F – «2»** – $< 60\%$ от общей суммы обязательных баллов

Содержание

1. Исполнение программы в ОС
2. Введение в Docker
3. Ликбез по Linux
4. Микросервисная архитектура
5. Немного о Kubernetes
6. Немного об Ansible
7. Немного о Terraform

Как исполняется программа в ОС?

- Операционная система – управляет ресурсами вычислительного узла и предоставляет API *системных вызовов* для программ
  - Поток – единица диспетчеризации для разделения CPU (int main())
  - Виртуальная память – пространство динамически выделяемой памяти RAM (+swap) для программы (malloc/free)
  - Сокет (ip + порт) – канал для обмена данными в рамках I/O

Как исполняется программа в ОС?

- Операционная система – управляет ресурсами вычислительного узла и предоставляет API *системных вызовов* для программ
  - Поток – единица диспетчеризации для разделения CPU (int main())
  - Виртуальная память – пространство динамически выделяемой памяти RAM (+swap) для программы (malloc/free)
  - Сокет (ip + порт) – канал для обмена данными в рамках I/O
- Что происходит, когда мы запускаем программу в Linux?
  - Создание подпроцесса (fork) – рождение нового PID как копии родительского процесса
  - Загрузка исполняемого файла в RAM для нового процесса (exec, секции ELF)
  - При этом был резервирована память под stack, а динамическая память (heap) будет выделяться по мере необходимости с помощью системных вызовов
  - Работа с памятью файлами, динамическими библиотеками, сокетами и т.п. – все через системные вызовы ОС (calling conventions)

Какие неявные зависимости есть у программы?

- Порядок сборки – библиотеки, пакеты, интерфейсы и API ОС
- Исполняющая среда – непосредственно ОС, виртуальная машина, интерпретатор и т.п.
- Файлы, появляющиеся в процессе работы программы
  - Stateful файлы – конфигурация, данные и т.п.
  - Временные и мусорные файлы – AppData, /tmp, /.cache и т.п.
- Сетевое окружение
  - Открытые порты (сокеты) и их forwarding наружу
  - Доступные IP-адреса сторонних сервисов и способ их адресации с помощью DNS

Содержание

1. Исполнение программы в ОС
2. Введение в Docker
3. Ликбез по Linux
4. Микросервисная архитектура
5. Немного о Kubernetes
6. Немного об Ansible
7. Немного о Terraform

Что такое Docker и зачем он нужен?

Docker – платформа для исполнения приложений в изолированных *контейнерах*.

- Файл Dockerfile – декларативное описание процесса сборки образа (image) для контейнера
- В результате сборки образа по Dockerfile мы получаем *образ* (image), или несколько образов в случае multi-stage
- Мы можем запустить контейнер на основе образа, указывая дополнительные параметры:
  - Отображение портов внутри контейнера на порты хоста (forwarding)
  - Режимы сети (host, bridge, none) для обеспечения сетевой связности между контейнерам, хостом и внешним миром
  - Volumes – можно примонтировать директории на хосте внутрь контейнера для обмена данными между ними (данные не удалятся, когда контейнер будет удален)
  - Переменные окружения (environment) для передачи конфигурации внутрь контейнера

Пример Dockerfile

```
# ARG - Use build-time variables.
ARG RELEASE
# LABEL - Add metadata to an image.
LABEL org.opencontainers.image.ref.name=ubuntu
# ADD - Add local or remote files and directories.
ADD file ... in /
# CMD - Specify default commands.
CMD ["/bin/bash"] # Specify default commands.
# ENV - Set environment variables.
ENV JAVA_HOME=/opt/java/openjdk
# RUN - Execute build commands.
RUN /bin/sh -c set -eux;
# COPY - Copy files and directories.
COPY --chmod=755 entrypoint.sh /__cacert_entrypoint.sh
# ENTRYPOINT - Specify default executable.
ENTRYPOINT ["/__cacert_entrypoint.sh"]
```

Пример Dockerfile (продолжение)

```
# CMD - Specify default commands.
CMD ["jshell"]
# EXPOSE - Describe which ports your application is listening o
EXPOSE [25565/tcp]
# VOLUME - Create volume mounts.
VOLUME [/data]
# WORKDIR - Change working directory,
WORKDIR /data
# STOPSIGNAL - Specify the system call signal for exiting a con
STOPSIGNAL SIGTERM
# HEALTHCHECK - Check a container's health on startup.
HEALTHCHECK &{["CMD-SHELL" "mc-health"] "30s" "0s"
```

Как собрать образ и запустить контейнер?

```
# . - директория, где лежит Dockerfile, т.е. текущая
docker build -t minecraft-server .

# посмотреть локальные образы
docker image ls

# -d - запуск в фоновом режиме (detached)
# -p - port forwarding: отображаем порт 25565 на хост
docker run -d -p 127.0.0.1:25565:25565 minecraft-server

# посмотреть контейнеры
docker ps
```

Пример docker-compose.yaml

```yaml
services:
  minecraft: # название контейнера (сервиса)
    image: itzg/minecraft-server:latest # имя образа и тэг
    pull_policy: daily # как часто обновлять образ
    tty: true
    stdin_open: true
    ports: # отображаем внутр. порт 25565 на порт хоста 25565
      - "25565:25565"
    environment: # конфигурируем через переменные окружения
      EULA: "TRUE"
      VERSION: "1.21.11"
      SERVER_NAME: "Уютные зимние приключения"
      DIFFICULTY: normal
    volumes: # монтируем директорию на хосте внутрь
             # контейнера для сохранения данных
      - ./data:/data
```

Как собрать образ и запустить контейнер?

```
# в директории, где лежит docker-compose.yaml
docker compose up -d

docker compose stop # (1) просто остановим все
docker compose down # ИЛИ (2): еще и грохнем контейнеры и т.п.
```

Discussion: что для нас Docker?

- Docker - среда виртуализации для исполнения программ в изолированных контейнерах
- Внутри каждого контейнера запущена своя ОС (обычно Linux)
  - Ядро ОС резидентно, т.е. должно лежать в оперативной памяти хоста, поэтому важен размер
  - Alpine Linux – 5 MB по образу
- Docker свойственно захватывать ресурсы хоста
  - RAM - эмулирует виртуальную память для контейнера, выделяя её из общей памяти хоста
  - Firewall - Docker управляет iptables в Linux (использовать совместно с ufw - не стоит)
- Какие плюсы и минусы Docker?
  - (+) Декларативная абстракция для описания сред для сборки и запуска программ
  - (-) Ненулевые издержки на RAM

Docker vs Виртуальная машина

- Виртуальная машина (VM):
  - Полная эмуляция оборудования – запущена отдельная ОС с собственным ядром
  - Медленный старт – стартует как отдельный компьютер
  - Полная изоляция – VM не видят друг друга на уровне ОС
- Docker контейнер:
  - Легковесная виртуализация – все контейнеры используют одно ядро хоста (namespaces + cgroups)
  - Малый overhead – контейнеры занимают мегабайты, не gigabytes
  - Изоляция на уровне процессов – видн друг друга в сети, но не при доступе к файловой системе
- Вывод: Docker идеален для микросервисов, VM – для полной изоляции и различных ОС

Содержание

1. Исполнение программы в ОС
2. Введение в Docker
3. Ликбез по Linux
4. Микросервисная архитектура
5. Немного о Kubernetes
6. Немного об Ansible
7. Немного о Terraform

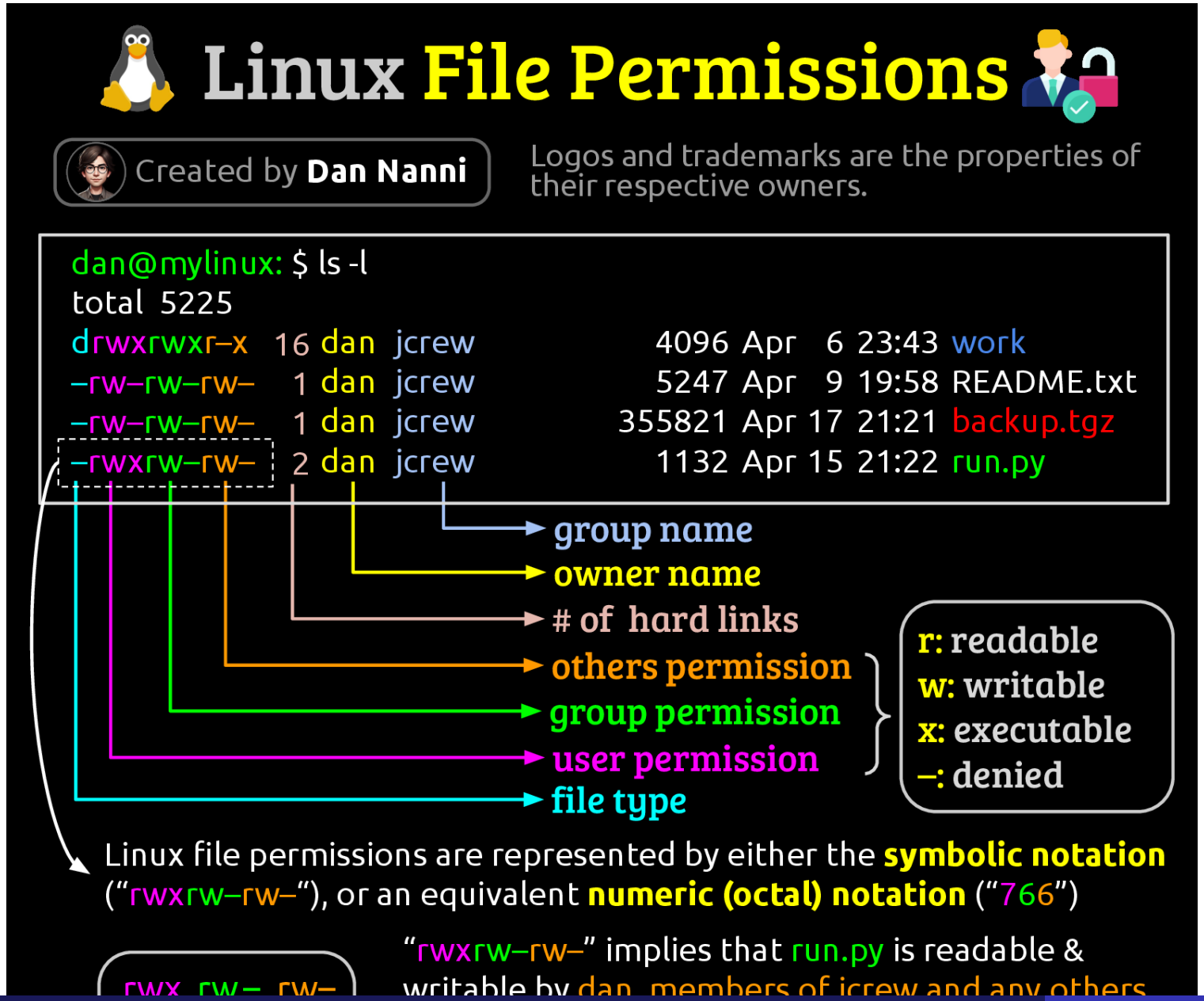
Какие свойства мы имеем в виду, говоря о Linux?

- Свободный, модульный и открытый код ОС и программ (copy-left лицензия GPL)
- Де факто промышленный стандарт: POSIX-совместимость, стандартные C-headers, ELF, .so и т.п.
- Декларативные способы решения задач по системному администрированию (в т.ч. DevOps)

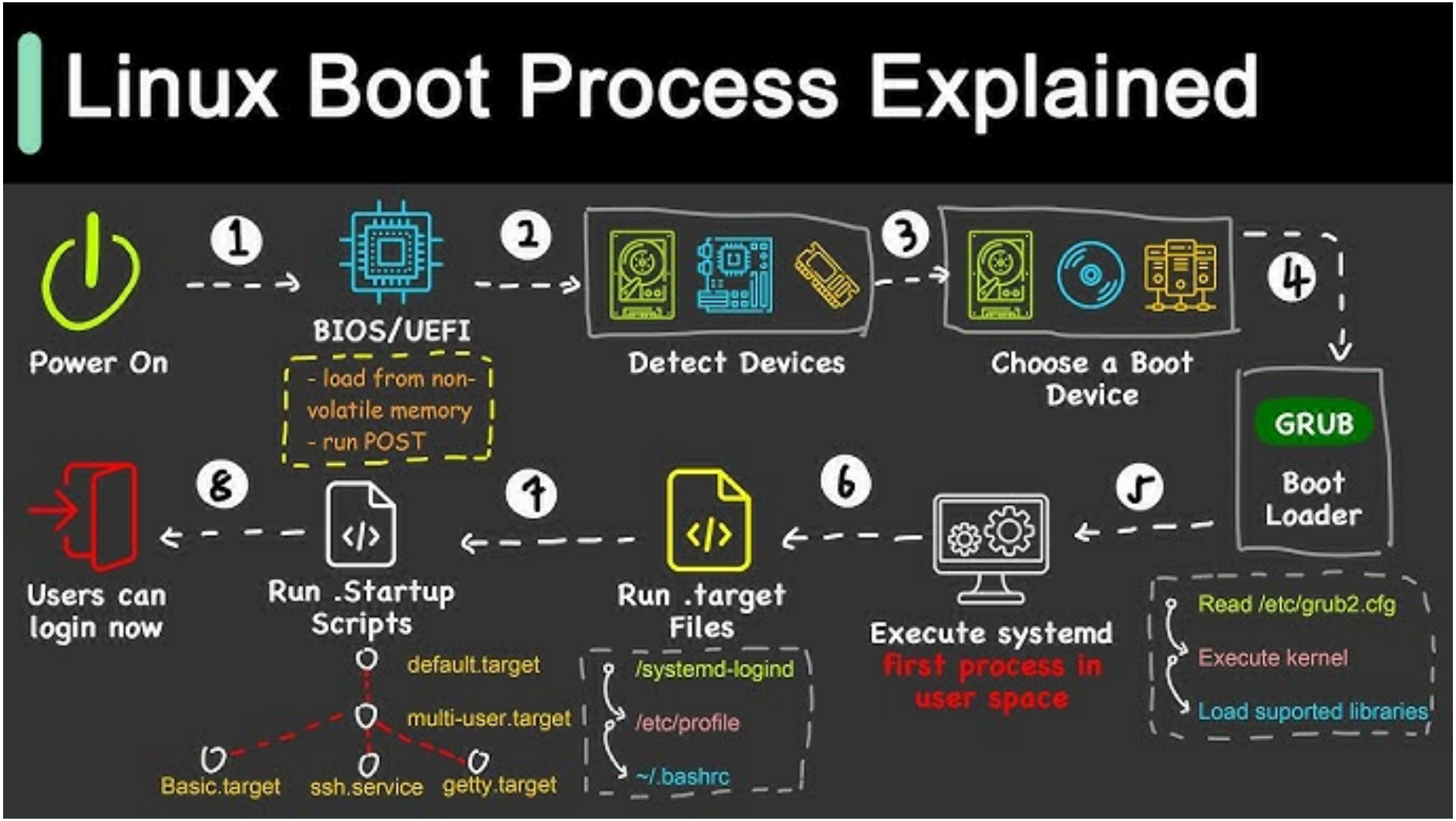
Что нужно обязательно представлять о Linux?

- Файловая система в Linux-системах
  - Все – это файл (включая устройства, сокеты и т.п.)
  - Иерархическая структура файловой системы (root, /home, /dev, /proc и т.п.)
  - Узлы иерархической структуры - inodes, ссылаются на блоки данных
  - Права доступа к файлам: владелец (u), группа (g), остальные (o); чтение (r), запись (w), выполнение (x)
- Порядок инициализации и управления ОС
  - Базово о физическом запуске - BIOS/UEFI, загрузчик (GRUB), ядро ОС, init-система (systemd)
  - Service в Linux: юнит, который может быть запущен, остановлен, перезапущен и т.п. с помощью systemctl
  - Процессы и демоны – top, htop, ps, systemctl status и т.п.
  - Пользователи и группы - useradd, groupadd, passwd и т.п.

Управление доступом к файлам



Загрузка сервисов



SSH-клиент

- SSH (Secure Shell) – протокол для безопасного удалённого доступа к машинам
  - Аутентификация: пароль или публичный ключ (более безопасно)
  - Шифрование трафика: все данные передаются в зашифрованном виде
  - Туннелирование: можно проксировать трафик через SSH (port forwarding)
- Подключение к серверу:
  - `ssh user@host` – базовое подключение
  - `ssh -i /path/to/key user@host` – с использованием приватного ключа
  - `ssh -p 2222 user@host` – подключение на нестандартный порт
- Генерация пары ключей:
  - `ssh-keygen -t ed25519 -f  /.ssh/id_ed25519`
  - Копирование публичного ключа на сервер: `ssh-copy-id -i /.ssh/id_ed25519.pub user@host`

SSH-сервер

- Конфигурация в `/etc/ssh/sshd_config`:
  - `Port 22` – порт для слушания (лучше менять на нестандартный)
  - `PermitRootLogin no` – запретить вход под root (обязательно!)
  - `PasswordAuthentication no` – разрешить только аутентификацию по ключам
  - `PubkeyAuthentication yes` – включить аутентификацию по публичным ключам
  - `PermitEmptyPasswords no` – запретить пустые пароли
  - `X11Forwarding no` – отключить X11 forwarding для безопасности
- Запуск и управление:
  - `systemctl restart sshd` – перезапустить сервис после изменения конфига
  - `systemctl status sshd` – проверить статус
  - `ss -tlnp | grep ssh` – проверить, что сервер слушает на нужном порту
- Хранение публичных ключей клиентов: `/.ssh/authorized_keys`

Упражнения (опциональные)

1. Создать папку (mkdir, -p) с файлами (touch, nano) и гранулярно настроить права доступа к ним (chmod, chown) под произвольную бизнес-задачу (например, для файлов веб-сервера - менять можем мы, но не он)
2. Создать service (service-файл) в Linux, который будет запускаться в качестве демона проверки состояния системы (скрипт на Bash) и логировать раз в какое-то время результаты в файл (systemctl list-dependencies, status, logs и т.п.)
3. Установить Docker, запустить произвольный сервис с помощью docker-compose:
   - nginx-сервер (проверить можно, открыв в браузере http://localhost:80)
   - Postgres (проверить, подключившись через psql, DataGrip и т.п.)
   - Minecraft-сервер (проверить можно, если есть игра)

Содержание

1. Исполнение программы в ОС
2. Введение в Docker
3. Ликбез по Linux
4. Микросервисная архитектура
5. Немного о Kubernetes
6. Немного об Ansible
7. Немного о Terraform

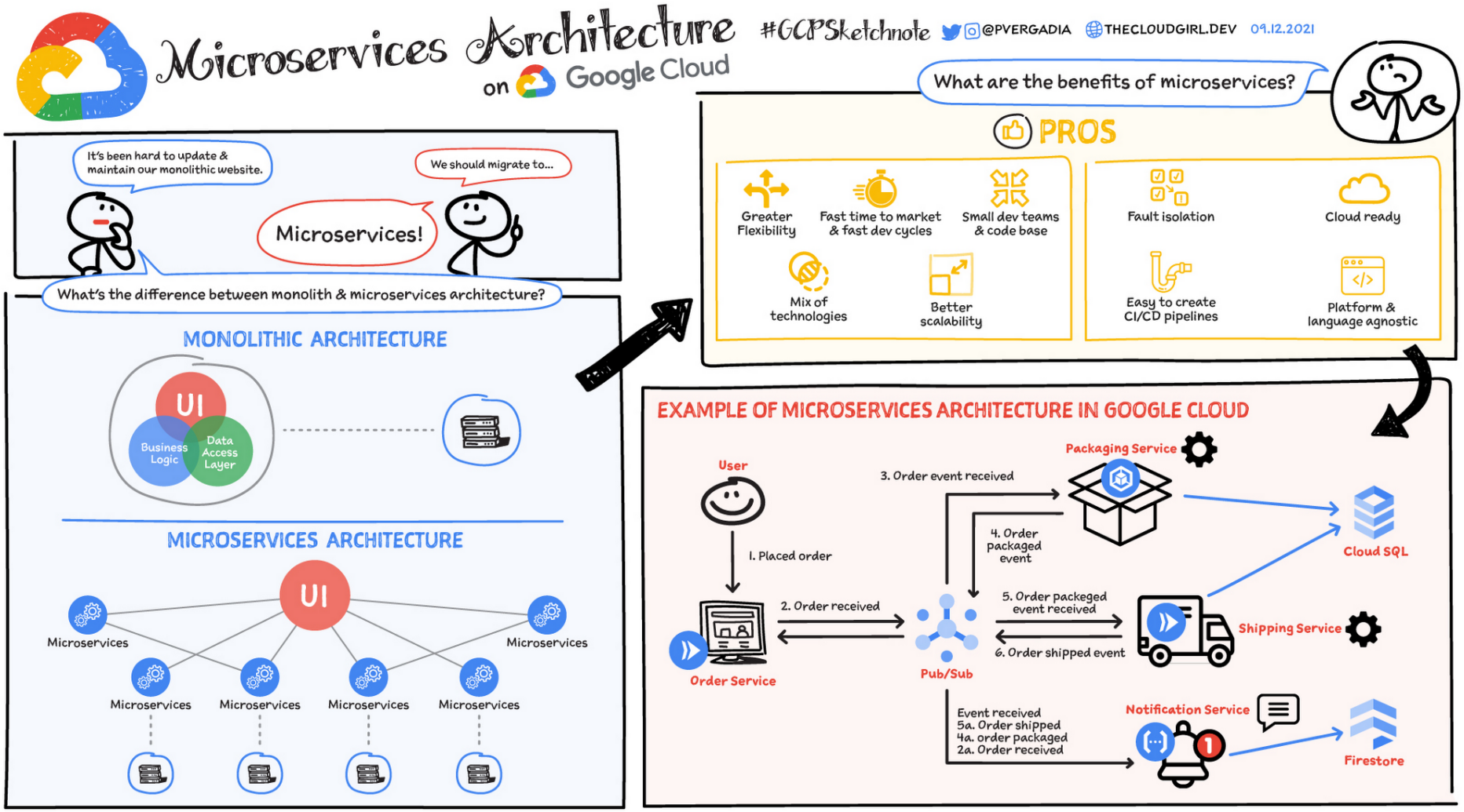
Что такое монолитная архитектура?

Монолитная архитектура – архитектурный стиль, при котором все компоненты приложения тесно связаны и развертываются как единое целое:

- (+) Простота разработки и развертывания – все в одном месте
- (-) Сложность масштабирования – масштабируем все целиком, даже если нужно масштабировать только часть
- (-) Сложность поддержки – изменения в одной части могут повлиять на другие части, что усложняет отладку и тестирование
- (-) Огромный монорепозиторий – трудно вмержить изменения, если команда большая
- (-) Фиксация на одном языке программирования и стеке

Например, 'Монолит' в VK – исполняемый файл (kPHP) размером в $\approx 1.5GB$

Постерный доклад на примере Google Cloud



Что такое микросервисная архитектура?

Микросервисная архитектура – архитектурный стиль, при котором приложение декомпозируется на микросервисы:

- Микросервис – автономный сервис, соответствующий одной бизнес-функции (со своим хранилищем)
- Разрабатывается независимо от других микросервисов, хорошо масштабируется
- Соединяется с другими микросервисами через HTTP/REST, gRPC, шины сообщений (Kafka, RabbitMQ) и т.п.

Микросервисы как узлы распределенной системы

- Экземпляры микросервиса – **поды** – горизонтально масштабируются в зависимости от внешней нагрузки
- Состав подов динамический – может меняться в процессе работы, что требует динамической маршрутизации и балансировки нагрузки
- Обязательно найдутся поды, которые будут работать некорректно (crash, memory leak, network issues и т.п.), поэтому требуется отказоустойчивость, а распределенные алгоритмы должны это учитывать

Содержание

1. Исполнение программы в ОС
2. Введение в Docker
3. Ликбез по Linux
4. Микросервисная архитектура
5. Немного о Kubernetes
6. Немного об Ansible
7. Немного о Terraform

Что такое Kubernetes?

Kubernetes (K8s) – система оркестрации контейнеров с открытым исходным кодом для автоматизации развертывания, масштабирования и управления контейнеризованными приложениями.

- Автоматическое развертывание подов на основе нагрузки и ограничений ресурсов
- Самовосстановление – перезапуск упавших контейнеров, замена недоступных подов
- Балансировка нагрузки и сетевая маршрутизация между подами (Service, Ingress)
- Хранение и управление секретами (пароли, токены, конфиги)
- Откат к предыдущим версиям (Deployment, ReplicaSet)

Архитектура Kubernetes

- Control Plane – управляет состоянием кластера:
  - API Server – REST API для управления ресурсами
  - etcd – распределённое хранилище конфигурации и состояния
  - Scheduler – распределяет поды на ноды
  - Controller Manager – контроллеры для управления Deployment, Service и т.п.
- Worker Nodes – исполняют поды:
  - kubelet – агент на ноде, взаимодействует с Control Plane
  - Container Runtime – Docker, containerd, CRI-O и т.п.

Пример манифеста Kubernetes (Deployment)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3 # запустить 3 экземпляра
  selector:
  matchLabels:
    app: nginx
  template:
  metadata:
    labels:
    app: nginx
  spec:
    containers:
    - name: nginx
    image: nginx:1.14.2
```

Основные команды kubectl

```
# Применить манифест
kubectl apply -f deployment.yaml

# Получить состояние подов
kubectl get pods -n default

# Просмотреть логи пода
kubectl logs <pod-name> -f

# Выполнить команду внутри пода
kubectl exec -it <pod-name> -- /bin/bash

# Просмотреть описание ресурса
kubectl describe deployment nginx-deployment

# Удалить ресурс
kubectl delete deployment nginx-deployment
```

Содержание

1. Исполнение программы в ОС
2. Введение в Docker
3. Ликбез по Linux
4. Микросервисная архитектура
5. Немного о Kubernetes
6. Немного об Ansible
7. Немного о Terraform

Что такое Ansible?

Ansible – инструмент для автоматизации управления конфигурацией и развертывания приложений на удалённых машинах.

- Agentless: не требует установки агента на целевых машинах, использует SSH
- Декларативный подход: описываем желаемое состояние, Ansible приводит систему в это состояние
- Идемпотентность: повторное выполнение playbook не вызывает нежелательных побочных эффектов
- Инвентарь (inventory) – список хостов и групп для управления
- Playbook – YAML-файл с последовательностью задач (tasks)
- Vault – для хранения секретов

Структура Ansible проекта

```
ansible-project/
|-- inventory.yaml          # список хостов
|-- playbook.yaml          # основной playbook
|-- roles/
|   |-- webserver/
|   |   |-- tasks/
|   |   |   `-- main.yaml  # задачи роли
|   |   |-- templates/     # шаблоны файлов
|   |   `-- vars/
|   |       `-- main.yaml  # переменные роли
|   `-- database/
|       `-- ...
`-- group_vars/
    `-- webservers.yaml    # переменные группы
```

Пример inventory.yaml

```yaml
all:
  children:
  webservers:
    hosts:
    web1:
      ansible_host: 192.168.1.10
      ansible_user: ubuntu
    web2:
      ansible_host: 192.168.1.11
      ansible_user: ubuntu
  databases:
    hosts:
    db1:
      ansible_host: 192.168.1.20
      ansible_user: ubuntu
```

Понятия в playbook.yaml

- **Task** – атомарная единица работы, которая выполняет одно действие (установка пакета, копирование файла, запуск команды и т.п.)
  - Задачи группируются в playbook или роль
  - Каждая task может иметь условия (when), циклы (loop) и зависимости (dependencies)
- **Role** – переиспользуемый набор файлов (tasks, handlers, templates, vars, files и т.п.) для выполнения конкретной функции
  - Инкапсулирует логику управления конфигурацией (например, роль `webserver` содержит все необходимое для установки и настройки веб-сервера)
  - Позволяет избежать дублирования кода и облегчает переиспользование в разных playbook'ах
- **Handler** – специальный вид task, который выполняется только при срабатывании события `notify`
  - Обычно используется для перезапуска сервисов при изменении конфигурационных файлов
  - Гарантирует, что сервис перезапустится только один раз, даже если несколько task'ов вызовут один и тот же handler

Пример playbook.yaml

```yaml
---
- name: Deploy web application
  hosts: webservers
  become: yes
  roles:
  - webserver
  tasks:
  - name: Install nginx
    apt:
    name: nginx
    state: present
  - name: Copy nginx config
    template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    notify: restart nginx

  handlers:
  - name: restart nginx
    service:
    name: nginx
    state: restarted
```

Основные команды Ansible

```
# Выполнить ad-hoc команду на всех хостах
ansible all -i inventory.yaml -m ping

# Выполнить playbook
ansible-playbook -i inventory.yaml playbook.yaml

# Выполнить playbook в режиме сухого прогона (check)
ansible-playbook -i inventory.yaml playbook.yaml --check

# Выполнить с повышенными привилегиями (become)
ansible-playbook -i inventory.yaml playbook.yaml -K

# Фильтровать по тегам
ansible-playbook -i inventory.yaml playbook.yaml -t deploy
```

Содержание

1. Исполнение программы в ОС
2. Введение в Docker
3. Ликбез по Linux
4. Микросервисная архитектура
5. Немного о Kubernetes
6. Немного об Ansible
7. Немного о Terraform

Что такое Terraform?

Terraform – инструмент Infrastructure as Code (IaC) для описания и управления облачной инфраструктурой декларативным способом.

- Provider – плагин для взаимодействия с облачным провайдером (AWS, GCP, Azure, Yandex Cloud и т.п.)
- Resource – примитивная единица инфраструктуры (VM, сеть, диск, RDS и т.п.)
- State – текущее состояние инфраструктуры (хранится локально или в удалённом backend)
- Plan – предварительный просмотр изменений перед применением
- Apply – применение плана для создания/изменения/удаления ресурсов

Основные концепции Terraform

- **Provider** – плагин для взаимодействия с облачным провайдером или сервисом (AWS, GCP, Azure, Yandex Cloud, Kubernetes и т.п.)
  - Определяется в блоке `provider` с указанием региона, учётных данных и других параметров
- **Resource** – объект инфраструктуры, управляемый провайдером (виртуальная машина, сеть, хранилище, база данных и т.п.)
  - Каждый ресурс имеет тип (например, `aws_instance`) и имя для идентификации
- **Variable** – входной параметр конфигурации Terraform для параметризации и переиспользования кода
  - Может иметь тип (`string`, `number`, `list`, `map`), значение по умолчанию и описание
- **Output** – выходное значение, которое выводится после применения конфигурации (IP адреса, идентификаторы ресурсов и т.п.)
  - Используется для передачи значений между модулями или для вывода информации пользователю
- **Ingress** – ресурс Kubernetes для управления входящим трафиком к сервисам внутри кластера

Пример конфигурации Terraform

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cb
  instance_type = "t2.micro"
  vpc_security_group_ids =
    [aws_security_group.web.id]
  tags = { Name = "web-server" }
}

resource "aws_security_group" "we
  name = "web-sg"
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Переменные и выходные значения в Terraform

```
variable "instance_count" {
  type        = number
  default     = 2
  description = "Количество инстансов"
}

variable "instance_type" {
  type    = string
  default = "t2.micro"
}

output "instance_ips" {
  value       = aws_instance.web[*].private_ip
  description = "Приватные IP адреса инстансов"
}
```

Основные команды Terraform

```
# Инициализация рабочей директории
terraform init

# Форматирование конфигурации
terraform fmt

# Валидация синтаксиса
terraform validate

# Просмотр плана изменений
terraform plan

# Применение плана
terraform apply

# Вывести текущее состояние
terraform show
```

Module — переиспользуемые компоненты

- Module – инкапсулированный набор ресурсов Terraform для повторного использования
- Локальные модули – в пределах проекта (./modules/vpc, ./modules/rds)
- Удалённые модули – из Terraform Registry или git-репозитория
- Параметризация – передача входных переменных в модуль, получение выходных значений

Пример использования модуля:

- `module "vpc"  source = "./modules/vpc", cidr_block = "10.0.0.0/16"`
- Обращение к выходу: `module.vpc.vpc_id`

Упражнения (опциональные)

1. Поднять локальный minikube-кластер, запустить Docker-контейнеры с ssh-агентами, настроить их через Ansible (hosts, playbooks)
2. Реализовать terraform-конфигурацию для настройки локального kubernetes-кластера для нескольких подов с nginx-балансировщиками
3. Снимаем для себя VPS-сервер с помощью хостинга и пишем для него ansible-конфигурацию для развертывания всего, что нам хочется (обращаем внимание на секреты и безопасность)

Итоги

- Обсудили инфраструктурные аспекты исполнения узлов в распределенных системах
- Выдано первое домашнее задание о часах и алгоритмах синхронизации